DEPARTMENT OF MATHEMATICS & COMPUTER SCIENCE

Faculty of Science and Technology · National University of Lesotho

CS3520

Computer Organisation and Architecture I

Week 6 — The Single-Cycle Processor I: Building the Datapath

Mr K. O. Setetemela

Course Coordinator · Department of Mathematics & Computer Science

Academic Year 2026/2027 · Semester A

RECAP & THIS WEEK

From Writing Instructions to Building the Machine

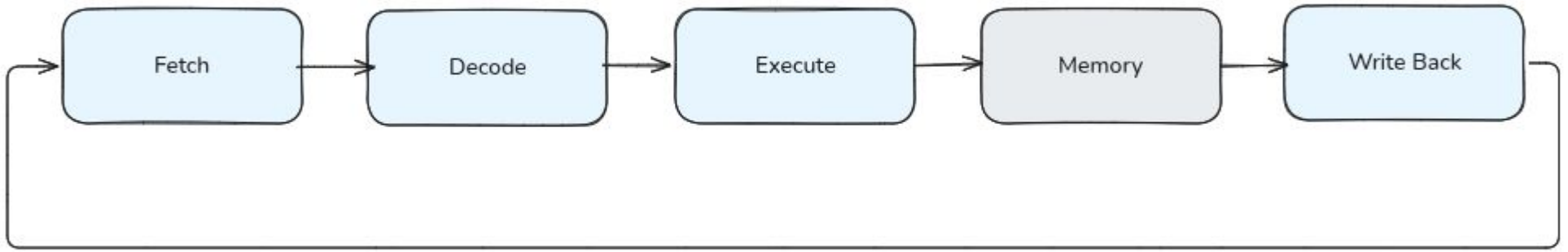
Weeks 3–5: you learned what the processor must do — the instruction set it obeys. From this week, you build the hardware that obeys it.

1	**The Job of a Processor**	What every instruction has in common
2	**Logic Design Conventions**	Combinational vs sequential; the clocking discipline
3	**The Instruction Fetch Datapath**	PC, instruction memory, +4 adder
4	**The R-Type Datapath**	Register file and ALU
5	**The Load/Store Datapath**	Sign extension and data memory
6	**The Branch Datapath**	Comparison and PC-relative target
7	**Composing the Single-Cycle Datapath**	One machine, four instruction classes

THE COMMON PATTERN

What Every Instruction Does

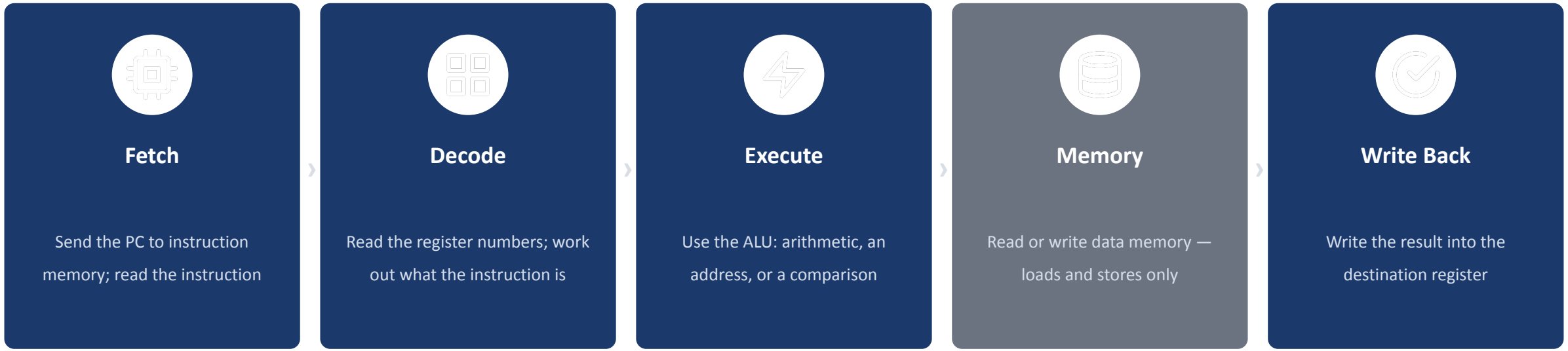
Whatever the instruction, the processor repeats the same cycle. Our datapath must support all of it, every cycle.



THE COMMON PATTERN

What Every Instruction Does

Whatever the instruction, the processor repeats the same cycle. Our datapath must support all of it, every cycle.



The Memory stage is greyed because only loads and stores use it — yet in a single-cycle machine, every instruction must still wait as though it did. Hold on to that observation; it is what eventually kills this design.

SCOPE

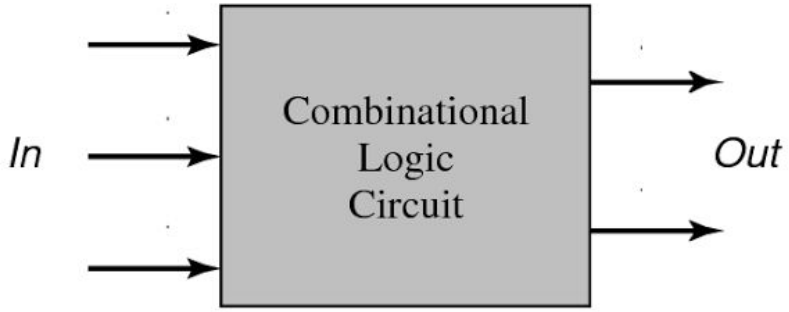
The RISC-V Subset We Will Build

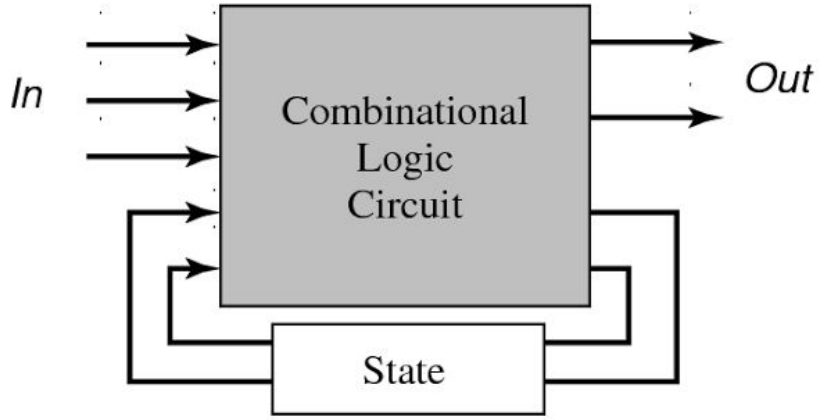
A processor that handles these four classes handles the essential shape of the whole ISA. Adding the rest is engineering, not new principle.

Class	Instructions	Format	What the datapath must provide
Memory reference	`lw, sw`	I-type / S-type	Register + sign-extended offset → address
Arithmetic / logical	`add, sub, and, or`	R-type	Two registers → ALU → destination register
Conditional branch	`beq`	B-type	Compare two registers; PC-relative target
(Week 7) Immediate	`addi`	I-type	Added once the control unit is complete

This is a 32-bit RV32 machine throughout: registers are 32 bits, the PC is 32 bits, and lw/sw move words. Keep that consistent — mixing 32-bit and 64-bit conventions is the single most common source of confusion in this topic.

Combinational and Sequential Elements





Combinational

`output = f(inputs)`

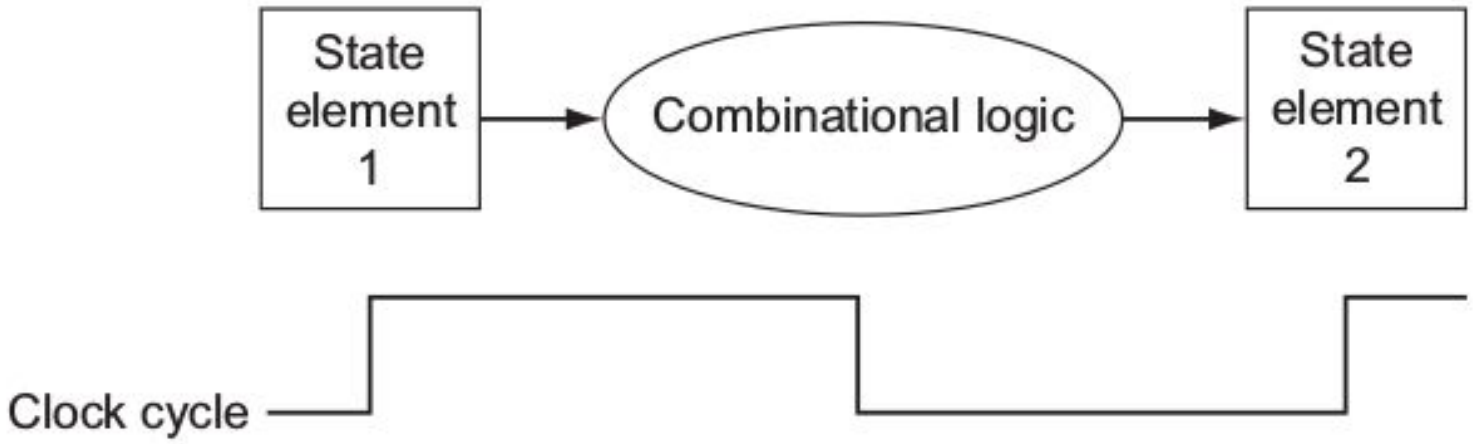
The output depends only on the inputs present right now. No memory of the past. Examples: the ALU, adders, multiplexers, sign-extension unit.

Sequential (State)

`output = f(inputs, previous state)`

Holds a value across time, updated on a clock edge. Examples: the program counter, the register file, instruction and data memory.

Combinational and Sequential Elements

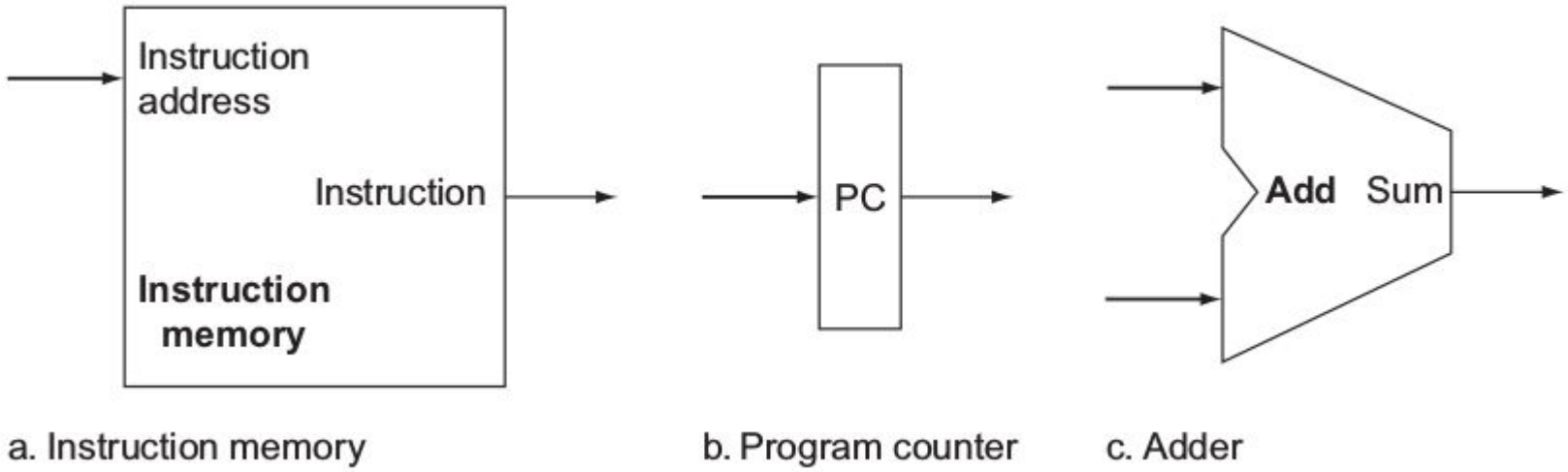


Edge-triggering allows a state element to be read and written in the same clock cycle.

However, the clock cycle must be long enough so that the input values are stable when the active clock occurs.

Instruction Fetch

Every instruction begins the same way: read the instruction at the PC, and work out the next PC.

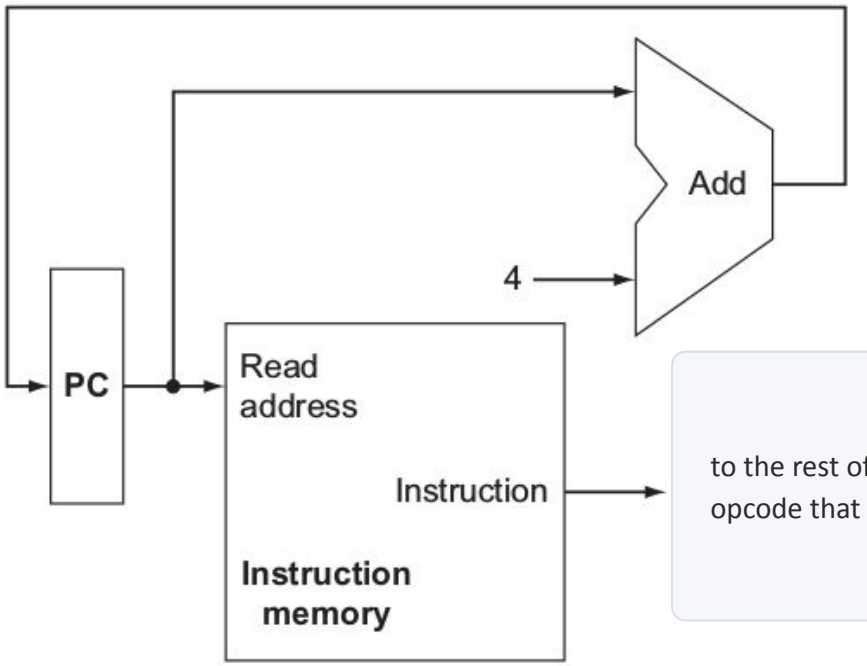


a. Instruction memory b. Program counter c. Adder

DATAPATH, STEP 1

Instruction Fetch

Every instruction begins the same way: read the instruction at the PC, and work out the next PC.



to the rest of the datapath — register numbers, immediate fields, and the opcode that drives control

IN-CLASS INQUIRY

?

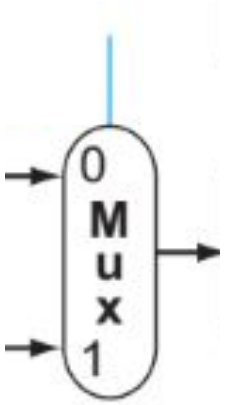
Why a dedicated adder rather than use the ALU for the addition operation?

R-Type: Register File, Multiplexor and ALU

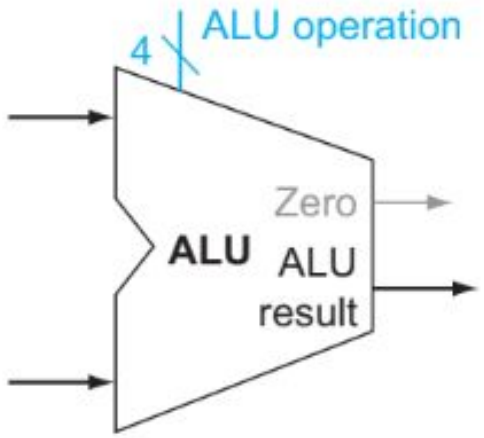
add, sub, and, or — read two registers, compute, write one register.



a) Register file



b) Multiplexor



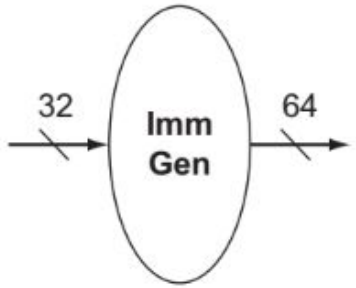
c) ALU

Memory Reference: Data Memory and Immediate Generation

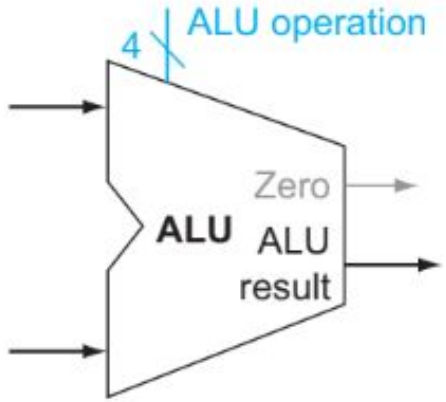
lw, sw — computes the memory address using register value and immediate field, read/write from/to data memory.



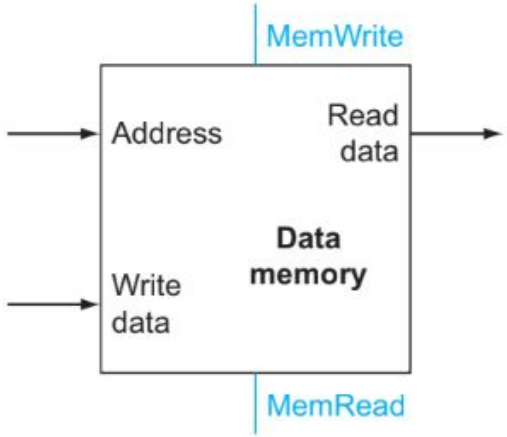
a) Register file



b) Immediate generation unit



c) ALU



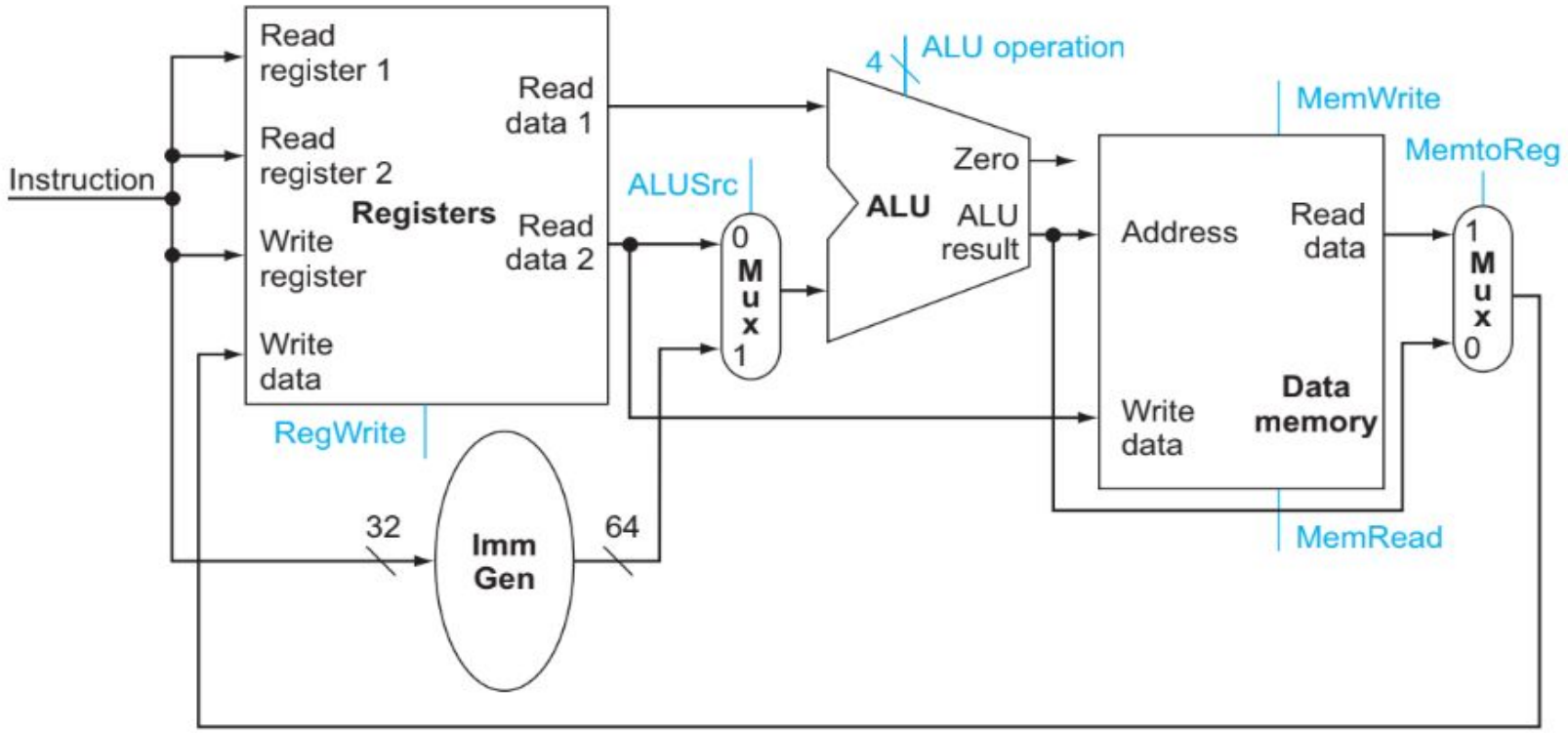
d) Data memory

DATAPATH, STEP 2

R-Type and Memory Reference

add, sub, and, or — read two registers, compute, write one register.

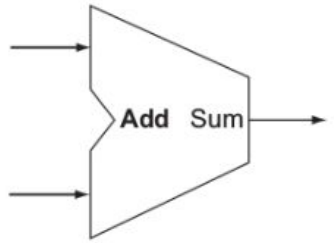
lw, sw — computes the memory address using register value and immediate field, read/write from/to data memory.



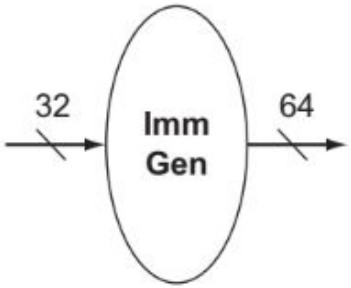
The register file is read and written in the same cycle. Edge-triggering makes this safe: reads use the old value, the write lands on the closing edge.

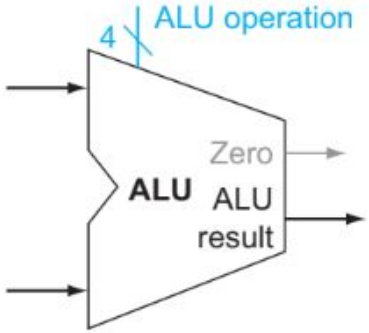
Conditional Branch - shift, adder

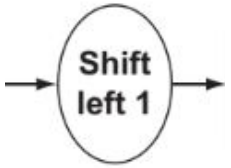
beq — evaluates the branch condition, computes PC-relative branch address.





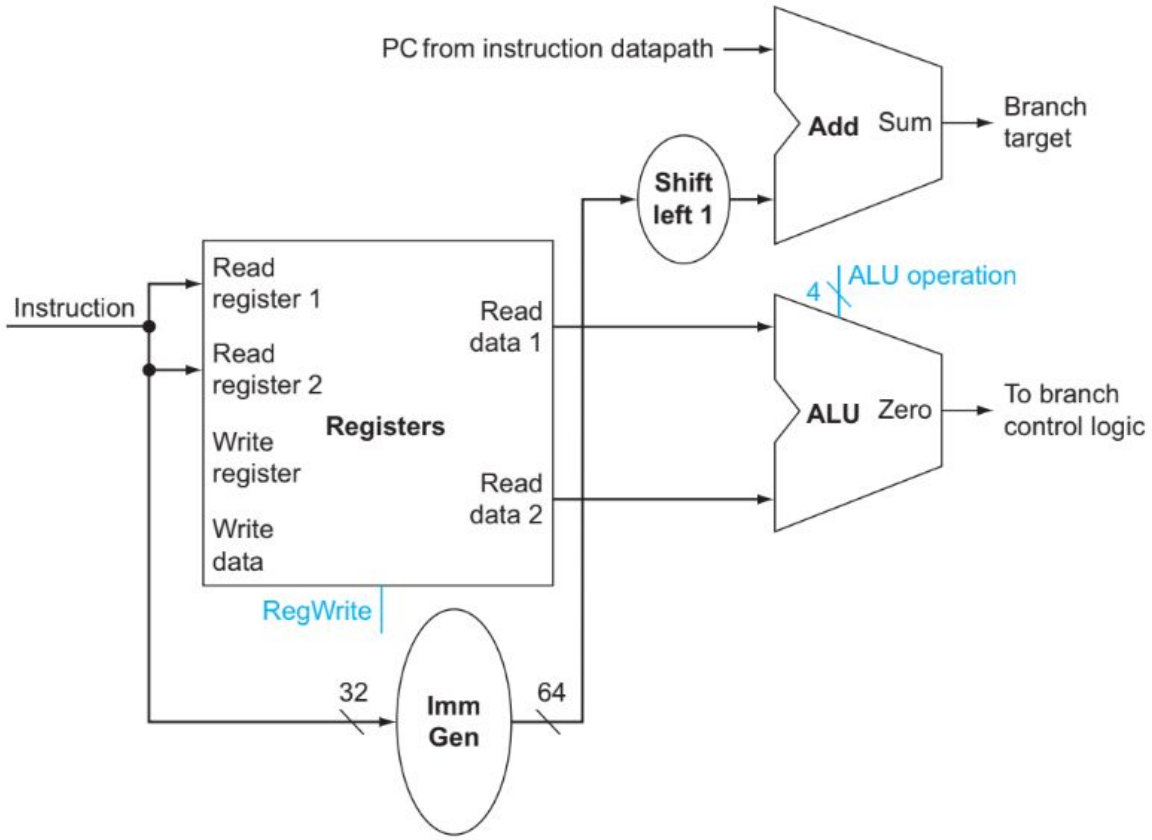




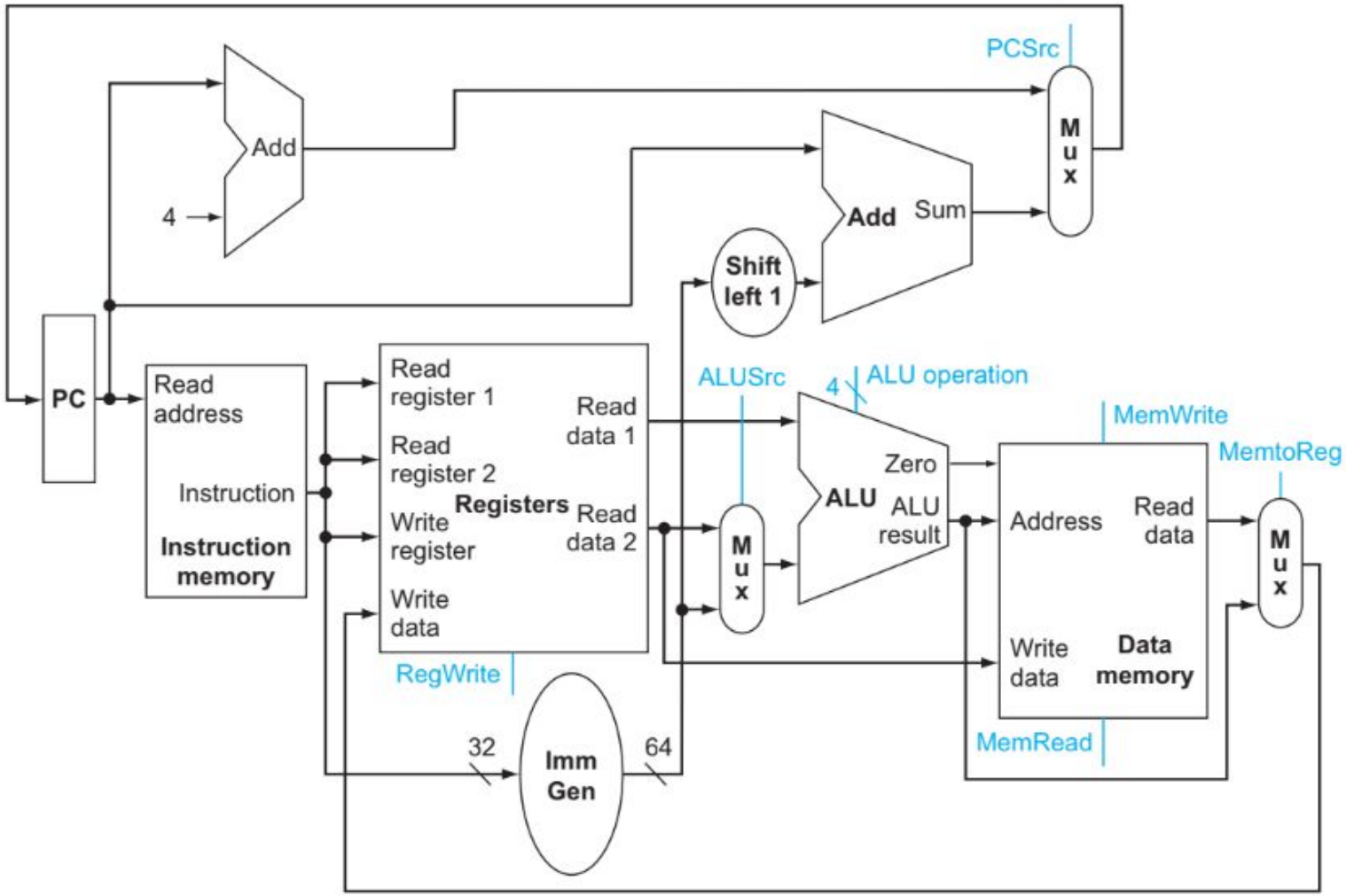


Conditional Branch

beq — evaluates the branch condition, computes PC-relative branch address.



Combined Datapath



THINK ABOUT IT

Every instruction in this machine takes exactly the same time — the time needed by the slowest one.

An add needs no memory access at all, yet it waits as though it did. What does that cost us? And if you could redesign the clock so each instruction took only the time it actually needed, what new problem would you create?

Discuss with a neighbour for two minutes before we continue.

THIS WEEK

Lab 5 & Class Test 1

Build the fetch datapath

PC, instruction memory and the +4 adder, wired in Logisim-evolution.

Add the register file and ALU

Extend it to execute R-type instructions end to end.

Test with real instructions

Load an encoding by hand and watch it flow through your circuit.

CLASS TEST 1 — THIS WEEK

Invigilated, closed book, Monday 5 - 6pm.

- Performance analysis: CPI, execution time, Amdahl's Law
- The RISC-V ISA: registers, instructions, addressing
- Instruction formats and encoding, in both directions
- Single cycle processor datapath

Covers Weeks 1– 6

CHECKPOINT · CLO3

Key Takeaways

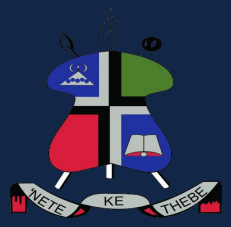
1. Distinguish combinational from sequential elements, and explain why an edge-triggered discipline lets a register be read and written in one cycle.

2. Draw the datapath required by each instruction class: fetch, R-type, load/store and branch.

3. Explain the role of every multiplexer in the combined datapath — what it chooses between, and which instruction class needs each choice.

4. Explain why a single-cycle machine has CPI = 1 but a clock period set by its slowest instruction.

Essential reading: Patterson & Hennessy, RISC-V Edition, Ch. 4 (§4.1–§4.3) • Stallings, Ch. 14



QUESTIONS?

COMING UP — WEEK 7

The Single-Cycle Processor II — Control

Control signals, the ALU control unit, extending the datapath, and why single-cycle fails

Mr K. O. Setetemela • ko.setetemela@nul.ls